

Explicación de la estructura del código y decisiones de implementación

El proyecto fue desarrollado en Python utilizando un enfoque modular para separar claramente las distintas etapas del pipeline de Reinforcement Learning aplicado al trading de BTCUSDT. La implementación se centra en el diseño del entorno de simulación y la formulación del problema como un Sequential Decision Process bajo el marco de Powell, mientras que el algoritmo de aprendizaje (PPO) es proporcionado por Stable Baselines3.

La estructura general del código se divide en cinco componentes principales:

1. Carga y preprocesamiento de datos
2. Ingeniería de variables (features)
3. Definición del entorno de trading
4. Entrenamiento y evaluación walk-forward
5. Generación de métricas y resultados

1. Carga y preprocesamiento de datos

La primera parte del código se encarga de cargar el dataset histórico de BTCUSDT y prepararlo para el entrenamiento.

Función `load_data()`

Esta función:

- Lee el archivo CSV original
- Corrige nombres de columnas
- Convierte la columna de fecha al formato `datetime`
- Ordena cronológicamente las observaciones
- Convierte las variables numéricas al tipo `float`

Decisión importante

Se eliminaron explícitamente filas inválidas (dropna) después de convertir las fechas para evitar inconsistencias temporales durante el entrenamiento.

Decisión no obvia

El dataset original contiene datos horarios, pero posteriormente se realiza un resampleo a velas de 4 horas. Esto se hizo porque:

- las velas de 1 hora generaban demasiado ruido,

- el agente tendía a sobre-operar,
- y la señal estadística era extremadamente débil.

Reducir la frecuencia temporal permitió estabilizar parcialmente el comportamiento del agente.

2. Ingeniería de variables (Feature Engineering)

La función `create_features()` construye el estado informacional del agente utilizando únicamente información histórica disponible al tiempo t .

Las variables utilizadas incluyen:

- retornos logarítmicos de corto y mediano plazo,
- medias móviles,
- desviaciones respecto a medias móviles,
- volatilidad histórica,
- rango high-low,
- z-scores de volumen y actividad de trading.

Decisión importante

Todas las variables fueron diseñadas para evitar lookahead bias. Ninguna feature utiliza información futura.

Alternativas consideradas

Se evaluó incluir:

- precios absolutos,
- indicadores técnicos adicionales,
- y ventanas más largas de momentum.

Sin embargo, estas alternativas fueron descartadas debido a:

- fuerte no estacionariedad,
- incremento en dimensionalidad,
- y ausencia de mejora observable durante pruebas iniciales.

Decisión no obvia

Las variables de volumen y trade count fueron normalizadas mediante rolling z-scores en lugar de utilizar valores crudos. Esto permitió:

- reducir sensibilidad a cambios estructurales de escala,
- mejorar estabilidad numérica,
- y facilitar el aprendizaje del agente.

3. Entorno de trading (BTCTradingEnv)

El núcleo del proyecto es la clase BTCTradingEnv, implementada usando Gymnasium.

Esta clase define:

- el espacio de estados,
- el espacio de acciones,
- la dinámica de transición,
- y la función de recompensa.

Espacio de acciones

El espacio de acciones es discreto:

- 0 → no operar
- 1 → posición larga
- 2 → posición corta

Decisión importante

Se eligió un espacio discreto en lugar de uno continuo porque:

- PPO convergía de manera más estable,
- el entrenamiento era más rápido,
- y el sizing continuo generaba políticas extremadamente inestables.

Lógica de trading

Cuando el agente abre una posición:

- se registra el precio de entrada,
- la posición permanece abierta hasta max_holding,
- y posteriormente se calcula el retorno neto incluyendo costos de transacción.

Decisión no obvia

La simulación utiliza una formulación semi-Markov:

- la recompensa no se entrega en cada timestep,
- sino únicamente al cierre de la operación.

Esto mejora la asignación de crédito en comparación con recompensas densas por paso temporal.

Reward Function

La recompensa se calcula usando:

$$\text{reward} = \text{net} \times 10$$

donde:

- net representa el retorno neto después de fees,
- y el factor de escalamiento amplifica la señal para PPO.

Alternativas consideradas

También se probaron:

- rewards tipo Sharpe ratio,
- log-returns por timestep,
- penalizaciones explícitas por drawdown,
- y rewards usando tanh.

Sin embargo:

- Sharpe ratio generaba problemas no Markovianos,
- rewards densos introducían demasiado ruido,
- y penalizaciones complejas dificultaban la convergencia.

4. Entrenamiento y evaluación

La evaluación se realiza mediante un esquema walk-forward de cuatro folds.

Cada fold:

- entrena un modelo PPO independiente,

- utiliza únicamente información pasada,
- y evalúa completamente out-of-sample.

Algoritmo PPO

Se utilizó PPO debido a:

- estabilidad relativa,
- facilidad de implementación,
- y compatibilidad directa con Gymnasium.

Los hiperparámetros principales incluyen:

- learning rate = $3e-4$
- gamma = 0.99
- batch size = 64
- n_steps = 512
- ent_coef = 0.1

Decisión importante

Se incrementó el número de timesteps hasta 100,000 porque con valores menores el agente colapsaba frecuentemente hacia políticas triviales de “no operar”.

5. Métricas y evaluación

Para cada fold se calculan:

- retorno total del agente,
- benchmark BTC buy-and-hold,
- benchmark SP500,
- Sharpe ratio,
- maximum drawdown,
- número de trades.

Decisión no obvia

El benchmark SP500 se incluyó para contrastar el desempeño del agente no solo contra el activo subyacente (BTC), sino también contra un mercado financiero tradicional con distinta estructura de riesgo.

Problemas observados durante el desarrollo

Durante el proyecto surgieron varios problemas típicos de Reinforcement Learning financiero:

1. Colapso hacia políticas triviales

En múltiples experimentos el agente aprendía:

- no operar nunca,
- o ejecutar siempre la misma acción.

Esto refleja la dificultad de extraer señal estadística robusta en mercados financieros.

2. Sobre-operación

En otros casos el agente realizaba cientos de trades sin rentabilidad positiva.

Los costos de transacción erosionaban rápidamente cualquier ventaja estadística marginal.

3. Sensibilidad al régimen de mercado

El comportamiento del agente variaba drásticamente entre folds:

- algunos periodos producían pocos trades,
- otros generaban trading excesivo,
- y los retornos cambiaban significativamente según el régimen.

Esto evidencia problemas de generalización y no estacionariedad.

Conclusión técnica

La implementación demuestra que el principal desafío en Reinforcement Learning financiero no es construir un agente funcional, sino diseñar un entorno capaz de producir señales explotables de forma robusta bajo condiciones realistas de mercado.

Aunque el sistema logra entrenar correctamente modelos PPO y ejecutar políticas activas de trading, los resultados muestran que:

- los mercados presentan baja predictibilidad,
- los costos de transacción afectan significativamente,
- y pequeñas decisiones de diseño en el entorno tienen gran impacto sobre el comportamiento aprendido.

Por ello, el proyecto enfatiza más la formulación rigurosa del problema y la evaluación crítica del sistema que la obtención de rentabilidad positiva.